Standalone build test: agentic reproducibility report

Placeholder Author
Placeholder affiliation
(Dated: September 25, 2026)

I. MAIN-TEXT PLACEHOLDER

Placeholder text standing in for the body of the paper. The two figures below carry the labels that the reproducibility subsection and the appendix refer to in the real document (the optimiser-convergence figure and the t-SNE figure).

A. Reproducibility report using agentic AI

To test and to sharpen the claims above, the work reported here was reproduced independently, in a clean room, by an agentic AI system. The agents received the text and figures of this submission as a PDF, the companion paper [1] — the source of the finite-difference optimiser and of the input angle encoding, neither of which the present submission states — and the public Iris and PhysioNet MIT-BIH [2] data; after the first round they were also sent the ECG recording behind the denoising figure, which the submission itself does not identify. They received *no* code: no implementation, no trained parameters, no clone of the game, no run scripts.

What they returned is a tested Python package, publicly available at `https://github.com/tobiasosborne/prototype-learning`, in which every figure and every table is regenerated by a script from a recorded run directory carrying its random seed, the commit hash and the simulator backend. A second density-matrix simulator, written blind by a different agent, serves as the numerical oracle; it agrees with the reference implementation to 10^{-12} on random circuits, and agreement between backends is always checked on full output distributions, never on the output marginal alone.

Table I is the scoreboard. Of the nine items that can be regenerated from the text of the submission, six reproduce: the activation surface of the prototype neuron, the bit-count target histogram and its convergence, the Iris overfitting contrast, perfect Flappy play on the agents' own clone of the game, and the ECG denoising table to within about 2 dB on a synthetic signal of the same form. The Iris accuracy comes out 2–4 points short of the published $\approx 98\,\%$, with no advantage of either architecture over 20 seeds. The ECG table carries a qualification: rerun on our own recording, supplied to the agents later, only the one-neuron cell of Table I reproduces — and there the prototype advantage is larger than published (2.8 dB against 0.2 dB) — while the two- and three-neuron prototype cells fall 2.5–3.2 dB short and the ordering at three neurons inverts; the full 7 700-sample recording gives the same shape (Appendix C 4). One does not: the weight picture of Fig. 2. In the agents' embedding both weight sets cluster away from the data; scored instead over 20 seeds against a criterion fixed in advance, trained prototype neurons are far more selective than classic ones but do not cover the data better, and their angles do not decode to data patterns. It is recorded as a discrepancy rather than tuned away, and Appendix C 2 states what was tried. The iterative baseline of Fig. 1 was a second such failure until the agents found that they had been grading it with the superposition D_{KL} while the scheme trains the per-input activations; in its own observable it reaches $\approx 10^{-3}$ on four seeds of six, and what remains assumed is the unit of the

FIG. 1. Placeholder for the main text's optimiser-convergence figure.

FIG. 2. Placeholder for the main text's t-SNE figure.

TABLE I. Scoreboard of the agentic clean-room reproduction. "Reproduced" means that the qualitative claim and the quantitative value — within the precision the published figure allows — both come out; "discrepancy" means that they do not, and Appendix A states what was tried.

Item	Published value	Clean-room reimplementation	Status
Activation surface, one prototype neuron	$(0,1)\to\max$, $(1,0)\to 0$	1.000/0.000/0.500/0.500 at the four corners	reproduced
Bit-count target histogram	bar heights	(number of ones + 1)/112, all 32 bars within 0.0002	reproduced
Superposition training	$D_{\mathrm{KL}} \approx 0$ in ≈ 8 iterations	median < 0.005 at sweep 9; 5 of 6 seeds $< 10^{-3}$ at sweep 20	reproduced
Iterative baseline	$D_{\mathrm{KL}} \approx 0$ by ≈ 120 iterations	per-input $D_{\mathrm{KL}} < 10^{-3}$ for 4 of 6 seeds within 10^5 elementary steps (a fifth reaches 1.4×10^{-3}); the superposition D_{KL} is a different observable	reproduced in the per-input observable; iteration unit assumed
Iris learning curves	classic overfits, prototype does not	classic test loss rises after 1–4 sweeps; the prototype's falls monotonically; over 20 seeds the classic test loss rises on 8, the prototype's on 2	reproduced
Iris accuracy	$\approx 98\,\%$	95/91/98 % on three splits (encoding ceilings 97/99/99 %); over 20 seeds 94.3 ± 2.8 % (classic 93.9 ± 2.7 %), 96.4 % on the filtered set when ambiguous codes are dropped	2–4 points short; no architecture advantage
Iris weight picture (t-SNE)	prototype weights span the data	t-SNE: both weight sets cluster away from the data. Scored over 20 seeds: prototype neurons far more selective than classic (median z +18 vs +1.8), but cover the data less (+0.6 vs +2.2); angles do not decode to data patterns	discrepancy (criterion fixed in advance)
Flappy Bird	"plays perfectly"	110/110 pipes (the frame cap) on all 20 held-out game seeds	reproduced, own clone
ECG PSNR, 1/2/3 hidden neurons	32.9/40.00/40.04 dB	37.4/38.0/38.1 dB on a synthetic signal of the same form; 36.8/37.5/36.9 dB on the authors' own recording (37.8/37.8/36.3 dB on all 7 700 samples)	within ≈ 2 dB on the synthetic signal; on the original recording the 1-neuron cell reproduces, the 2- and 3-neuron cells fall 2–4 dB short

published "iteration" (Appendix A).

The agents then went beyond reproduction, and the appendix reports three extensions. A noise layer built from the published calibration data of seven quantum computers (IBM, Quantinuum, IonQ, Google, IQM, Rigetti) shows that gate noise at today's published error rates does *not* break any of these small networks. A compilation and cost study identifies the seven-qubit Flappy activation map on a trapped-ion machine as the first experiment worth submitting, and prices it. A shot-noise study shows that the binding constraint is the finite number of measurements rather than gate error, and that a sequential measurement rule cuts the Flappy shot budget by a factor of 4.6 at no loss of play. All of this is simulation: the noise models are matched to vendor calibration data, and nothing reported here has been run on a quantum computer.

[1] B. Rosenhahn, T. J. Osborne, and C. Hirche, Stochastic neural networks for quantum devices (2026), arXiv:2602.22241 [quant-ph].
[2] G. B. Moody and R. G. Mark, IEEE Engineering in Medicine and Biology Magazine **20**, 45 (2001).
[3] J. Kiefer and J. Wolfowitz, The Annals of Mathematical Statistics **23**, 462 (1952).
[4] N. Gleinig and T. Hoefler, in *58th ACM/IEEE Design Automation Conference (DAC)* (2021) pp. 433–438.
[5] A. Wald, The Annals of Mathematical Statistics **16**, 117 (1945).

Appendix A: Scope and method

This appendix records what the agentic clean-room reimplementation of Section I A produced: a **reproduction** of every figure and table of this submission that can be regenerated from its text; a **calibrated-noise study** of the seven quantum computers for which vendors publish calibration data, with a **compilation and cost study**; and a **shot-noise programme**, since the number of repeated measurements, not gate error, turns out to limit a hardware demonstration. Only the first part is a reproduction. The other two are extensions, and they are simulations parameterised by published calibration data — no circuit was submitted to a quantum computer.

Every number below comes from a *run directory*: a folder holding a script's output with the random seed, the commit hash and the simulator backend that produced it, named by time stamp (e.g. `bitcount_noise/20260820T145335Z`) and quoted throughout. Open choices are listed in Appendix H, calibration provenance in Appendix I. Two machines were used and every run directory records which: the August 2026 runs — the reproduction, the noise study and the hardware plan — on one workstation (64 cores, one RTX 3090); three reruns of 18 September 2026 (`bitcount/20260918T111515Z`, `ecg/20260918T132924`, `iris/20260918T085641Z-drop`) on a 12-thread laptop with no GPU; and the runs of 25 September (`20260925`) on the workstation again, from clean commits, reproducing every laptop number they overlap bit for bit. Wall times are quoted with the number of cores actually used. No run was allowed to exceed about two hours; noise-aware training of the 11-qubit bit-count network ($\approx$ 13 h for its grid) and the full ECG/Iris noise grid ($\approx$ 8 h) were therefore not run, and are reported as not run rather than approximated.

Appendix B: Notation, optimiser and simulators

a. Activation and encoding. A network has n_{in} input qubits, n_{hid} hidden qubits (one per hidden neuron) and one output qubit. Its output is the *activation* $P(\text{out} = 1)$: thresholded for a classifier (Flappy: flap iff $P(\text{out} = 1) > \frac{1}{2}$), taken as the predicted value for a regressor (ECG). A feature $s \in [0, 1]$ enters as $R_X(\theta(s))$. The submission does not state $\theta(s)$, but the companion paper [1] prints $CR_X(1.369)$ for $s = 0.4$ and $CR_X(1.7722)$ for 0.6, which pins

$$\theta(s) = 2 \arcsin \sqrt{s}, \qquad P(\text{qubit reads } 1) = s, \tag{B1}$$

with $\theta = \pi s$ kept as an ablation (Appendix C 4).

b. Loss, shots and margin. With inputs in superposition the measured joint distribution of (input string x, output bit) gives the *model distribution*

$$q(x) = \frac{P(\text{inputs} = x,\ \text{out} = 1)}{\sum_{x'} P(\text{inputs} = x',\ \text{out} = 1)}, \tag{B2}$$

the inputs conditioned on the output having fired, compared to the target p by $D_{\text{KL}}(p \,\|\, q)$ with q floored at 10^{-12}. For noise we also use $\text{TV}(p, q) = \frac{1}{2} \sum_x |p(x) - q(x)|$. Probabilities are estimated from N shots, so $\hat{p}$ has standard deviation $\leq 1/(2\sqrt{N})$; the *decision margin* of a thresholded output is $\delta = |P(\text{out} = 1) - \frac{1}{2}|$, and a frame decided from N shots is decided wrongly with probability $\approx \Phi(-2\delta\sqrt{N})$.

c. Noise and fidelity. A calibrated machine is three channels: depolarising, from the vendor's average gate error ε by $\varepsilon = p(1-2^{-k})2^k/(2^k+1)$ for a k-qubit gate; relaxation with the vendor's T_1, T_2, applied during every gate for its duration and over every idle interval, scheduled as early as possible as IBM's compiler does; and readout confusion with p_{01}, p_{10}. The implementation agrees with Qiskit Aer to 2×10^{-15} on a noisy three-qubit circuit. A *noise scale* s multiplies p, $1/T_1$, $1/T_2$ and the readout errors together, so $s=1$ is the published machine. For compiled circuits we also quote the first-order probability that no error occurred at all,

$$F \approx \prod_{\text{2q}}(1-\varepsilon_2)\prod_{\text{1q}}(1-\varepsilon_1)\prod_{\text{qubits}}F_{\text{idle}}(t)\prod_{\text{qubits}}\Big(1-\tfrac{p_{01}+p_{10}}{2}\Big), \tag{B3}$$

$$F_{\text{idle}}(t) = \tfrac{1}{2} + \tfrac{1}{6}(2e^{-t/T_2} + e^{-t/T_1}).$$

d. Optimiser and simulators. Neither paper gives the step sizes, the temperature schedule or what one "iteration" is, so these are the reimplementation's choice: central differences [3] with $a_n = a_0/(n+1)^{0.602}$, $c_n = c_0/(n+1)^{0.101}$ inside the Metropolis rule with geometric cooling. One *sweep* visits every parameter once in random order, evaluates the probes $x \pm c_n e_i$ and the proposed step and takes the best of the three to the Metropolis rule, at $3\times$ (parameters) circuit evaluations, 108 for the bit-count network. Whole-vector steps stalled near $D_{\text{KL}} \approx 10^{-2}$ at every setting tried; with a rotating mini-batch the incumbent loss must be re-evaluated every sweep (Appendix C 3). The reference simulator is plain NumPy and is the oracle for everything else: an independent density-matrix simulator written blind by a second agent agrees with it to 10^{-12} on random circuits, and a C density-matrix library, a trajectory simulator for circuits too large for density matrices (Iris at 17 qubits), a GPU backend and a batched state-vector kernel are each tested against it on *full output distributions*, never on $P(\text{out}=1)$ alone.

Appendix C: Reproduction of the four experiments

1. Bit-count

The target histogram is published only as a bar chart. Reading the bars gives $p(x) = (\text{ones in } x+1)/112$ for all 32 bars to within 0.0002, with $112 = \sum_x(\text{ones}+1)$. The network is the published one — 5 inputs, 5 prototype hidden neurons, one classic output, 11 qubits, 36 angles — trained so that Eq. (B2) matches that target with the inputs in equal superposition of all 32 strings.

Figure 3 gives $D_{\text{KL}} = 5.5\times10^{-4}$; the submission's own printed model histogram, read off the figure, has $D_{\text{KL}} \approx 2.6\times10^{-3}$ against the same target. In Fig. 4 (left) the median of six optimiser seeds crosses $D_{\text{KL}} = 0.005$ at sweep 9 and five of six are below 10^{-3} at sweep 20, one stalling at 0.014 (`bitcount/20260925T153548Z`, whose superposition series are bit for bit those of `bitcount/20260820T135930Z`). The published "≈ 0 in about 8 iterations" does not define an iteration; against these sweeps it matches.

a. The iterative baseline: the observable, not the scheme. Figure 1 also shows a slower iterative curve reaching ≈ 0 by about 120 iterations, described only as "optimizing examples iteratively"; the companion paper [1] gives sample-wise squared-error updates but no schedule. The earlier finding that such a construction stalls at $D_{\text{KL}} \approx 0.15$ was an observable mismatch rather than a failure of the scheme. A sample-wise scheme trains the *per-input activation* $P(\text{out}=1)$ on one random basis input per step, while it was being graded with the superposition D_{KL} of Eq. (B2). The two are different quantities, because the prototype block $R_X(\varphi)\,CR_X(\theta)\,R_X(-\varphi)$ does not restore the input register — the uncomputation acts on a control qubit that the CR_X has already entangled with the hidden qubit — so $P(\text{inputs}=x,\ \text{out}=1) \neq 2^{-n_{\text{in}}}P(\text{out}=1 \mid \text{inputs}=x)$. Measured on the archived run `bitcount/20260925T153548Z`: the networks trained on the per-input activations reach a per-input D_{KL} below 10^{-3} while their superposition D_{KL}, recorded alongside, never falls below 0.035 (Fig. 4, dashed). The two

observables are not interchangeable.

Two defects of the reimplementation's own construction were real and are fixed: the annealer offered the finite-difference probes $x \pm c_n e_i$ as acceptance candidates alongside the proposed step, which on a one-example objective degenerates into a greedy random walk; and the schedules for a_n, c_n and the temperature advanced once per *example*, so the temperature had underflowed by $n \approx 50$ while the probe amplitude never shrank. The corrected baseline offers only the Kiefer–Wolfowitz step to the Metropolis rule, moves one coordinate per iteration, and holds $a = 2$, $c = 0.05$, $T = 0$ fixed.

The published "iteration" of the red curve is not defined. Taking it to be one *elementary* Kiefer–Wolfowitz step — one coordinate, one input, four objective evaluations (two probes, the step, the re-scored incumbent), against the 36-coordinate sweep of 109 evaluations used before — the hint to use "1000 times more iterations" is consistent: $120 \times 1000 \approx 10^5$ steps. That reading is an assumption and is the discrepancy that survives here. Over 10^5 elementary steps on six seeds, with the per-input D_{KL} monitored every 1 000 steps (Fig. 4, right; run `bitcount/20260925T153548Z`), seeds 3, 2, 4 and 1 fall below 10^{-3} at 12 000, 23 000, 62 000 and 73 000 steps and end at 1.1×10^{-3}, 4.4×10^{-4}, 7.2×10^{-4} and 8.9×10^{-3} (minima 4.9×10^{-4}, 3.1×10^{-4}, 3.7×10^{-4}, 9.5×10^{-4}); seed 5 comes close (minimum 1.4×10^{-3}, final 2.4×10^{-3}) and seed 0 plateaus at 0.02–0.03 (minimum 1.7×10^{-2}); the published endpoint is $\approx 10^{-3}$. The superposition D_{KL} of these same networks never falls below 0.035 over the run (range 0.035–0.58, endpoints 0.07–0.53) — precisely the old "stall". The run took 44 minutes on six workstation cores, one seed per core; seeds 0–3 reproduce the earlier four-seed laptop run `bitcount/20260918T111515Z` bit for bit.

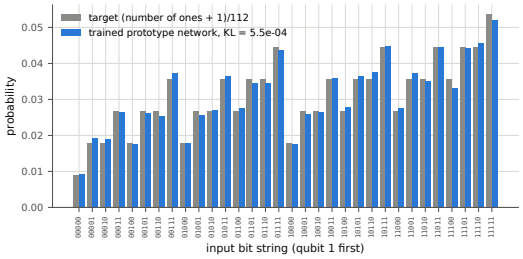


FIG. 3. Target histogram (grey) and the trained prototype network's model distribution (blue) over the 32 input strings; $D_{\mathrm{KL}} = 5.5 \times 10^{-4}$. Run `bitcount_noise/20260820T145335Z`, seed 1.

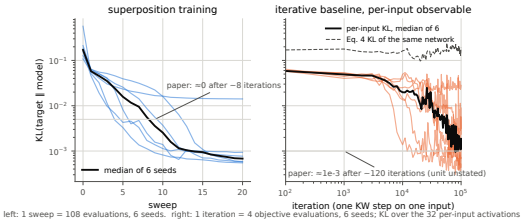


FIG. 4. Left: superposition training, D_{KL} of Eq. (B2) per sweep, six optimiser seeds and their median. Right: the iterative baseline in its own observable — the D_{KL} over the 32 per-input activations against the elementary iteration count (one coordinate, one input, four objective evaluations), six seeds; dashed, the superposition D_{KL} of the same networks, which does not follow it. Run `bitcount/20260925T153548Z` (both panels, six seeds; its superposition series are bit for bit those of `bitcount/20260820T135930Z`).

2. Iris

How the label enters the amplitude is not stated. The published bar heights — 1/86, 4/86, 9/86 on the training set and 1/154, 4/154, 9/154 on the test set, two independent integer normalisations — fix the weight of class ℓ as $\propto (\ell+1)^2$: the amplitude carries $\ell+1$ and the histogram its square, and no other exponent produces both normalisations. The split is not stated either; a stratified one-third training split (50 flowers) is used. Each feature is quantised into three k-means clusters, giving a 12-bit one-hot code per flower, and the training set is loaded as one superposition over its distinct codes; the network has 12 inputs, 4 hidden neurons and one output (17 qubits), against a classic-neuron network of the same size, with labels predicted by rounding each code's read-out weight.

In Fig. 5 the classic network drives its training loss to 10^{-5} within a few sweeps while its test loss bottoms out early and stays at 0.03–

0.04 — it memorises — whereas the prototype network descends more slowly on both and its test loss is still falling at sweep 40. Prototype test accuracy is 95 %, 91 % and 98 % on the three splits against the published $\approx 98\,\%$; the encoding caps accuracy at 97 %, 99 % and 99 %, since two flowers of different species can share a 12-bit code.

a. The weight picture, scored. Figure 2 is a two-dimensional (t-SNE) embedding in which the prototype weights are said to "span the data". The reimplementation's embedding does not show it — both weight sets cluster away from the data — and a t-SNE picture cannot be scored, so it was replaced by two measurements, with the pass criterion fixed before the seeds were run. The first reads what the hidden neurons *do*: from the probability $P_j(c)$ that neuron j fires on training code c it forms the *coverage* $\mathrm{mean}_c \max_j P_j(c)$ (every code has a neuron that fires on it) and the *selectivity* $\mathrm{mean}_j\,[\max_c P_j(c) - \mathrm{mean}_c P_j(c)]$ (each neuron fires on some codes far more than on the rest), each expressed as a z-score against 1 000 untrained layers drawn from the initialisation. The second reads the angles: it decodes each neuron's preferred 12-bit pattern and measures how well the patterns cover the training codes, as $\rho = 0$ for random patterns and $\rho = 1$ for the best possible choice. The figure counts as reproduced if, over 20 seeds, the prototype layer is positive on both firing scores and beats the classic layer on both (paired one-sided Wilcoxon test, $p < 0.01$), and, for the angles, if its median ρ is at least 0.5.

Over 20 seeds and both treatments of ambiguous codes (below), i.e. 40 training runs summarised in `iris_sweep/20260925T173530Z`, the prototype layer has the selectivity and not the coverage. Its selectivity z has median +18.0 against +1.8 for the classic layer ($p < 10^{-5}$ under both policies); its coverage z has median +0.6 against +2.2, i.e. the classic neurons fire on more of the data. The angle score is zero within error for both networks (medians -0.07 to $+0.14$, the bootstrap interval of every prototype median containing 0). Trained prototype neurons are specialised detectors, but they do not tile the data better than classic neurons, and their angles do not read as data patterns under either angle-to-bit convention. Figure 2 is recorded as not reproduced. The same sweep confirms the overfitting contrast of Fig. 5: the classic network's test loss rises after its minimum on 8 of 20 seeds, the prototype's on 2.

b. Ambiguous codes, and how labels are read out. The original preprocessing removed the binary patterns assigned to two classes (e-mail of 21 August 2026). Under this "drop" policy the training *and* test samples with such codes are removed, which conditions the test set on its own labels, so accuracy is quoted both on the filtered test set and on all 100 test flowers. The sweep also exposed a fragility of the read-out. Rounding "each code's weight to the nearest label" needs one overall scale, since the three class weights 1 : 4 : 9 are known only up to a constant; fitted afresh on each evaluated set, it makes a flower's predicted label depend on which *other* flowers are evaluated with it. On one seed, removing three ambiguous test codes moved that scale by a whole class step and the prototype network's accuracy from 94 % (all 100 flowers) to 39 % (the 87 kept). Reading each flower from its own basis state instead of from the superposition changes nothing (the two read-outs agree on every run of the sweep); fitting the scale once on the training set removes the effect. With that read-out, over 20 seeds, both networks reach 94 % on all test flowers (prototype 94.3 ± 2.8 and $94.1 \pm 2.6\,\%$ without and with dropping; classic 93.9 ± 2.7 and $94.5 \pm 2.4\,\%$) and 96.4 % on the filtered test set under dropping — below the published $\approx 98\,\%$, with no accuracy advantage for either architecture.

c. On the split. The published histograms show 15 non-empty bars for the training set and 30 for the test set. The 50-flower training splits used here give 14, 12 and 14 distinct codes, close to 15, but the 100-flower test splits give only 20, 20 and 18, not 30: three clusters per feature do not spread 100 flowers over 30 codes on any split tried. The figure supports the *direction* of the split, not its ratio, which remains an assumption. An earlier draft claimed the bar counts justified the ratio; an independent review caught the over-reading and the justification was withdrawn.

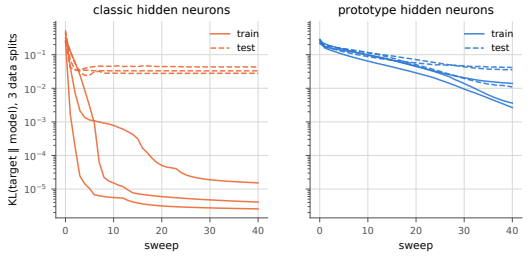


FIG. 5. Training (solid) and test (dashed) loss per sweep on three data splits, classic against prototype network. Runs `iris/20260820T144849Z`, `...T145454Z`, `...T145455Z`.

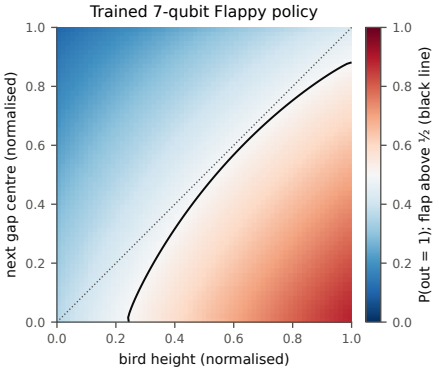


FIG. 6. The trained controller's activation $P(\text{out}=1)$ over the two observations; the black line is the decision boundary $P(\text{out}=1)=\frac{1}{2}$ (flap to its right), the dotted diagonal is "bird at gap height". Run `flappy/20260820T143753Z`.

3. Flappy Bird

The submission gives no reward, no training protocol and no game constants, so a deterministic, window-free clone was built. Its physics are a choice: the classic constants (impulse -9, gap 100, spacing 144) are provably unplayable by a memoryless two-feature policy — one flap overshoots by 40.5 px against a 38 px budget — so impulse -7, gap 120, spacing 180 are used. Height and the centre of the next gap enter two input qubits by Eq. (B1); four prototype hidden neurons and one classic output flap when $P(\text{out}=1)>\frac{1}{2}$, 17 angles in all. Training minimises the negative episode return (0.01 per frame, 1 per pipe) plus a tracking term $|\text{height}-\text{gap}|$ supplying a slope where every policy dies within 30 frames; 300 random vectors are scored on short episodes and the best seeds 25 sweeps on five 1500-frame episodes.

The trained network passes 110 pipes in 5 000 frames — the cap, a perfect game — on all 20 held-out game seeds, for all 8 optimiser seeds tried, in about three minutes of training (run `flappy/20260820T143753Z`); Fig. 6 shows the learned boundary.

4. ECG denoising

Five consecutive noisy samples enter five input qubits; one to three prototype hidden neurons and one output predict the clean centre sample as $P(\text{out}=1)$. The recording is not identified in the text; the published trace is a smooth two-beat signal of 1 800 samples in $[0,1]$ with additive noise of standard deviation ≈ 0.03. A synthetic ECG of McSharry form was generated to match it, Gaussian noise of $\sigma = 0.02$ added (noisy input 32.0 dB), the first half used for training and the second for test, and the experiment repeated on PhysioNet MIT-BIH record 100 [2].

On that synthetic signal PSNR is 37.4, 38.0 and 38.1 dB for one, two and three prototype neurons against the published 32.9, 40.00 and 40.04: within about 2 dB at two and three neurons, and 4 dB *above* the published value at one neuron (Fig. 7). The prototype network beats the classic one at one and two neurons and on every SSIM; the classic one wins at three. The alternative encoding $\theta=\pi s$ costs 0.6–7 dB here, independent evidence for Eq. (B1). On MIT-BIH record 100 (run `ecg/20260820T165912`) the same networks reach 36–38 dB.

a. The original recording. The recording behind the published Fig. 13 was then supplied by the authors (e-mail of 21 August 2026; two MATLAB files, identified by their SHA256 digests in the run directory and not redistributed). It arrives already noisy and clipped to $[0,1]$: over its first 1 800 samples — the published window — the measured noise standard deviation is 0.035 and

the input PSNR 27.0 dB, against the $\sigma = 0.02$ and 32.0 dB assumed above. Rerunning the identical protocol on it (Table II; run `ecg/20260925T153739Z-mailed1800`, identical to the earlier laptop run `ecg/20260918T132924` in every cell) splits Table I in two. The one-neuron result reproduces, and more strongly than published: the prototype network gains 2.8 dB over the classic one at one hidden neuron where the published table gains 0.2 dB, and it lifts a 27.0 dB input to 36.8 dB, so the smallest prototype network already does most of the denoising. The top of the table does not reproduce: the two- and three-neuron prototype cells fall 2.5 and 3.2 dB short, and at three neurons the ordering inverts (classic 38.02 dB against prototype 36.86 dB). Whole-signal SSIM is 0.997–0.999 in every cell and separates nothing; the 7-sample windowed SSIM, 0.84–0.93, ranks the cells as the PSNR does.

With the signal no longer a candidate explanation this is recorded as a discrepancy and not tuned. Three candidates remain: the noise level and the clipping of the supplied trace (a trace clipped to $[0, 1]$ is not the additive-Gaussian model the loss assumes); the restart and selection protocol, which the text does not give and which here is 8 restarts per cell with the best taken by training MSE; and the train/test split, which the text does not give either and which here is first half against second. The full 7 700-sample recording (input 27.5 dB; run `ecg/20260925T165741Z-mailedfull`, 39 minutes on 24 workstation cores) rules out the window as a fourth: it gives the same shape — prototype 37.75, 37.82 and 36.25 dB, classic 34.90, 37.72 and 38.19 dB, i.e. a 2.9 dB prototype advantage at one neuron, a tie at two and the classic network ahead at three.

Appendix D: The experiments under calibrated noise

1. Seven machines as noise models

Each vendor's published gate errors, gate durations, T_1, T_2, readout error and connectivity were collected into a device file whose every field carries its source URL, retrieval

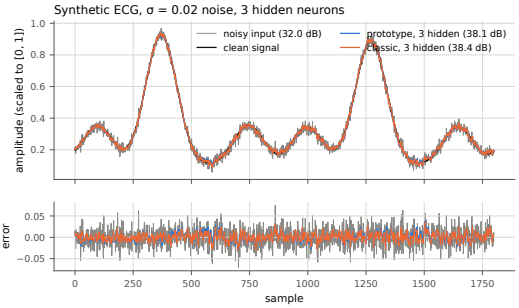


FIG. 7. Noisy input, clean signal and the two three-neuron networks' outputs (top); their errors (bottom). Run `ecg/20260820T162014`.

date (20 August 2026) and a verbatim quotation (Appendix I); Table III summarises them. Three caveats govern everything that follows. *Quantinuum publishes no gate durations and no T_1/T_2*: its idle error is charged from its published memory error per depth-1 circuit time (2.0×10^{-4} for H2-1, 2.2×10^{-4} for H1-1), and its simulated noise carries no idle error. *Rigetti publishes only a two-qubit error* for Ankaa-3, its 99.5 % headline being for a different gate than the universal one needed here, so every Rigetti number is an upper bound. *IQM's T_1/T_2 are a third-party snapshot*, and its T_2 without echo is 8.5 μs against the 21 μs echo value used. Where a vendor publishes nothing, the channel is modelled as error-free and flagged, not invented.

2. Bit-count

The trained 11-qubit networks were evaluated exactly, as density matrices on the GPU, under each preset (run `bitcount_noise/20260820T145335Z`; Fig. 8). The model distribution moves by TV = 0.005 (Quantinuum H2-1) to 0.039 (IQM Garnet), and the KL to the target rises from 5.5×10^{-4} to 7×10^{-4} on H2-1 and to 6×10^{-3} on Garnet. The prototype network is better under every preset, but the displacement *caused* by the noise, $D_{\mathrm{KL}}(\text{clean} \,\|\, \text{noisy})$, is the same for both (median 0.0021 against 0.0022): they have identical two-qubit gate counts, so the prototype's advantage under

TABLE II. Table I recomputed on the authors' own recording (first 1 800 samples, the published Fig. 13 window, noisy input 27.0 dB; and all 7 700 samples, input 27.5 dB). PSNR in dB on the test half. Whole-signal SSIM is 0.997–0.999 and windowed SSIM 0.84–0.93 in every cell, so neither is tabulated. The memo row is the synthetic-signal result above, whose input is a cleaner 32.0 dB and therefore not directly comparable. Runs `ecg/20260925T153739Z-mailed1800` and `ecg/20260925T165741Z-mailedfull`.

	CQNN (classic)			PTQNN (prototype)		
hidden neurons	1	2	3	1	2	3
published Table I	32.72	38.20	39.65	32.90	40.00	40.04
reimplementation, same recording	33.97	36.33	38.02	36.80	37.48	36.86
difference	+1.25	−1.87	−1.63	+3.90	−2.52	−3.18
reimplementation, full recording	34.90	37.72	38.19	37.75	37.82	36.25
memo: synthetic signal	—	—	—	37.4	38.0	38.1

TABLE III. The seven device presets (medians where the vendor publishes per-qubit tables). Two-qubit error ε_2, one-qubit error ε_1, readout error (mean of p_{01}, p_{10}), coherence times, two-qubit gate time. "—" = not published.

Machine	Connectivity	ε_2	ε_1	readout	T_1	T_2	2q gate (ns)
Quantinuum H1-1	all-to-all	9.7×10^{-4}	1.8×10^{-5}	2.3×10^{-3}	—	—	—
Quantinuum H2-1	all-to-all	1.1×10^{-3}	1.9×10^{-5}	1.0×10^{-3}	—	—	—
IonQ Forte	all-to-all	4.0×10^{-3}	2.0×10^{-4}	5.0×10^{-3}	55 s	1 s	950 000
Google Willow	square	4.1×10^{-3}	5.3×10^{-4}	8.0×10^{-3}	68 μs	89 μs	42
IBM Fez (Heron r2)	heavy-hex	3.9×10^{-3}	2.3×10^{-4}	7.7×10^{-3}	145 μs	88 μs	84
IQM Garnet	square	4.9×10^{-3}	8.0×10^{-4}	3.0×10^{-2}	27 μs	21 μs	30
Rigetti Ankaa-3	square	1.0×10^{-2}	—	—	—	—	72

noise is inherited from its better noiseless fit, not from robustness of the neuron type. Scaling IBM Fez's noise by s gives $D_{\mathrm{KL}} \propto s^{1.7}$ and TV $\propto s^{0.95}$.

Training *under* the noise works, on a 7-qubit version (3 inputs, 3 hidden; the 11-qubit grid would have cost 13 hours and was not run): it recovers the noisy KL by a median factor of 158 (prototype) and 361 (classic), Fig. 9. But the recovered parameters are *worse* on a noiseless simulator — the optimiser folds the channel into the angles instead of finding noise-robust ones.

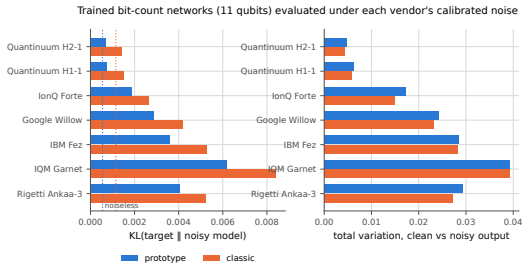


FIG. 8. The trained bit-count networks under each machine's noise model. Left: KL to the target, noiseless values dotted. Right: how far the output distribution moved. Rigetti's row models only its two-qubit error. Run `bitcount_noise/20260820T145335Z`.

3. Flappy Bird

For a thresholded activation what matters is not how far $P(\mathrm{out} = 1)$ moves but whether it crosses $\frac{1}{2}$. Evaluated exactly under every preset on its 65×65 decision grid and played for 20 held-out games (run `flappy_noise/20260820T154249Z`; Fig. 10), the controller *still plays perfectly on all seven machines*: only 1.5–4 % of visited frames are decided differently, because the noise contracts every activation toward $\frac{1}{2}$ by about the same

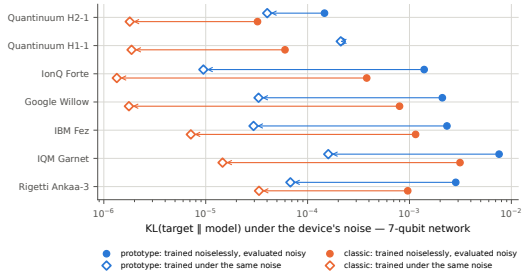


FIG. 9. Noise-aware training of the 7-qubit bit-count network: KL under each machine's noise for parameters trained noiselessly (filled) and with the same noise model in the loop (open), median of three seeds. Same run.

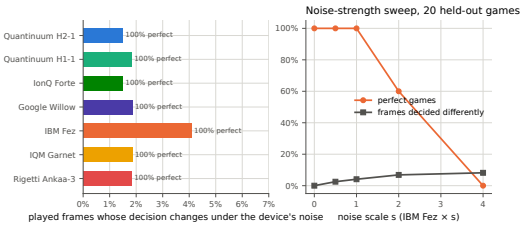


FIG. 10. Flappy under gate noise. Left: fraction of played frames whose decision changes under each machine's model; every machine still yields 100 % perfect games. Right: IBM Fez's noise scaled by s. Run `flappy_noise/20260820T154249Z`.

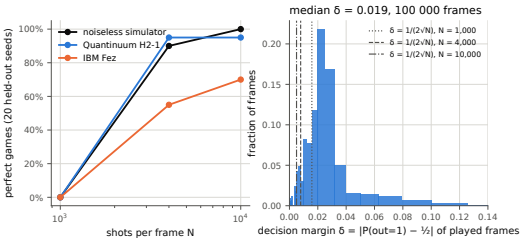


FIG. 11. Flappy under shot noise. Left: perfect games when every frame is decided from N shots. Right: distribution of the decision margin δ over the 100 000 frames the noiseless controller plays; vertical lines mark the margin that N shots can just resolve. Same run.

amount (mean $|\Delta P| = 0.002$–0.018) and so rarely changes the ordering. There is a factor of two of headroom: Fez's noise scaled by 2 leaves 60 % of games perfect, by 4 none.

Shot noise is a different matter (Fig. 11). Re-deciding every frame from N measurements, the controller wins no game at $N = 1\,000$, 90 % at 4 000 and all at 10 000 noiselessly; on Fez's model only 55 % at 4 000. The cause is the margin distribution over played frames: median $\delta = 0.019$, tenth percentile 0.004, first percentile 0.0002; the measured flip rate agrees with the average of $\Phi(-2\delta\sqrt{N})$ over it to 3 %. A deterministic shift of the boundary is tolerated far past a stochastic one — Fez at scale 2 decides 20 % of frames differently and still wins 60 % of games, while 1 000 shots decide only 8.5 % differently and win none —. Fine-tuning under Fez's noise for five sweeps changes nothing: the training episodes all survive already, so the objective has no slope.

4. ECG and Iris, reduced run

These two grids were run in a *reduced* configuration only (run `ecg_iris_noise/20260820T155618Z`): ECG on 8 test windows instead of 898 and two presets instead of seven; Iris with 16 Monte-Carlo trajectories per evaluation instead of 2 000 (per-outcome standard error ≈ 12 %), two presets, and perfect preparation of the dataset state assumed. The full grids cost 4–6 CPU-hours (ECG) and 8 hours (Iris), outside the budget. The results (Fig. 12) are indicative only: ECG loses −0.1 to +0.4 dB under Quantinuum H2-1 and 1.6–2.7 dB (classic) under IBM Fez, while the prototype cells *gain* up to 3.8 dB under Fez on eight windows — a small-sample artefact, not a finding. Iris keeps its 95 % accuracy under H2-1 and loses it under Fez (64 ± 44 % over two trajectory seeds) for the prototype network, whose read-out is a conditional distribution over 4 096 input strings and is therefore directly exposed to probability leaking out of the 20-code support. One noisy sweep of ECG fine-tuning gains 2.7 dB under Fez and 3.7 dB noiselessly: it finishes the clean optimisation, it does not fold the channel in.

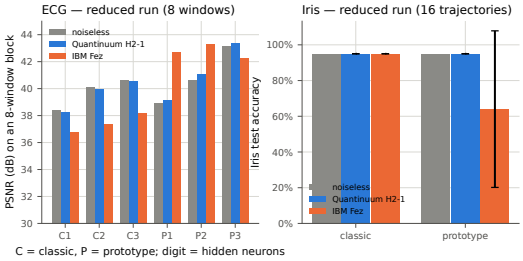


FIG. 12. Reduced-configuration noise runs, quoted for the record. Left: ECG PSNR on an 8-window block. Right: Iris test accuracy from 16 trajectories, error bars over two trajectory seeds. Run `ecg_iris_noise/20260820T155618Z`.

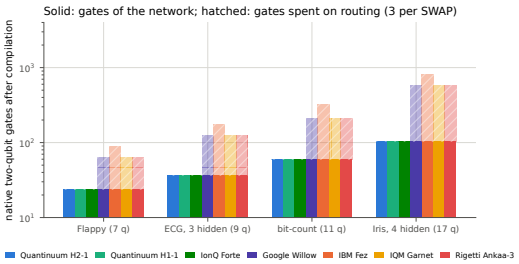


FIG. 13. Native two-qubit gates after compiling each network for each machine (greedy router, log scale). All-to-all machines need only the network's own gates; the lattices spend most of their budget on routing. Run `hardware-plan/20260821T084554Z`.

Appendix E: What it takes to run this on a real machine

All numbers here are printed by one script into `hardware-plan/20260821T084554Z` (file `numbers.md`); it joins every calibration number to its provenance entry and refuses to run if one is missing.

1. Compilation

Every prototype neuron touches every input, so the two-qubit gates form a complete bipartite graph between inputs and hidden qubits — the worst case for a lattice. On an all-to-all machine each controlled rotation costs exactly two native two-qubit gates: 24 for Flappy, 60 for bit-count. On a lattice the compiler must insert SWAPs at three native gates each; with a greedy router the bit-count circuit needs 327 native two-qubit gates on IBM's heavy-hex lattice and 207 on a square lattice, 82 % and 71 % of them SWAP-borne (Fig. 13). IBM's own compiler needs 97 at optimisation level 2, 3.4× fewer, so this router is a device-independent comparison instrument and not what one would submit. The Flappy circuit is 12.9 μs long on Fez (1.56 μs of it readout) and 17.9 ms on IonQ Forte, whose gates take a millisecond.

2. Expected fidelity

Figure 14 stacks the four factors of Eq. (B3) for the two smallest circuits, with the exactly simulated total-variation distance between the compiled circuit run clean and run under noise beside each bar. Flappy at 7 qubits is a high-fidelity circuit on trapped ions ($F \approx 0.92$ on H2-1, 0.83 on Forte), marginal on Willow and Fez (0.63; 0.46 with the greedy router, 0.75 with IBM's compiler) and poor on Garnet (0.24). Bit-count at 11 qubits is feasible on the H-series (0.81) and Forte (0.64), usable with IBM's compiler on Fez (0.48) and dead on Garnet (0.008), whose routed circuit lasts 86 % of its T_2. Idling is charged only over each qubit's schedule gaps, since relaxation during a gate is already inside the vendor's gate error; charging the whole circuit duration on every qubit is 10–15 points more pessimistic on the superconducting machines and is printed beside it in the run directory.

Two cautions. The simulated distances of the *compiled* circuits (0.03 on the H-series, 0.16 on Forte, 0.43 on Fez) are 5–15× larger than for the uncompiled circuits of Appendix D 2. And vendors' two-qubit errors are measured in isolation — IBM's own layered benchmark for Fez is 1.7× worse than its isolated gate error — so the lattice numbers are optimistic.

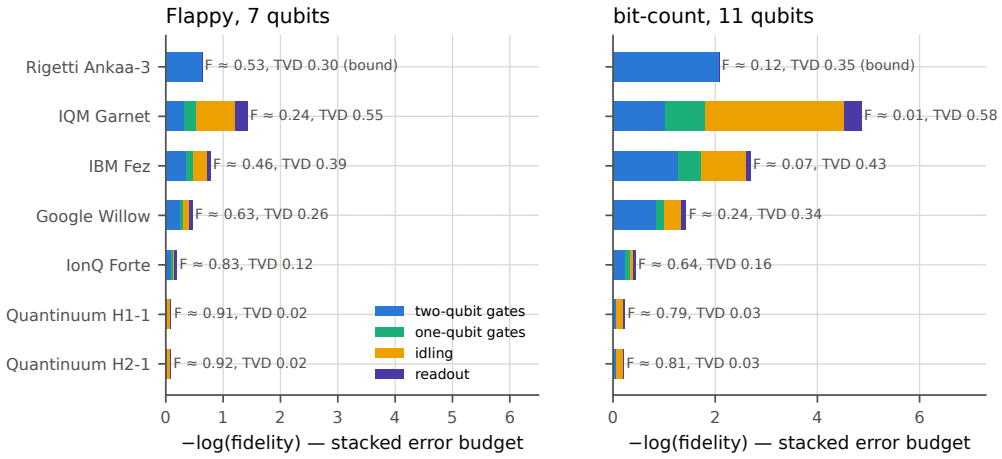


FIG. 14. Error budget per machine for the Flappy (left) and bit-count (right) circuits: the stacked bars are $-\log$ of each factor of Eq. (B3), so their total is $-\log F$; labels give F and the simulated total-variation distance of the compiled circuit. Quantinuum's idle term comes from its published memory error; Rigetti's is a bound from its two-qubit error only.

3. Shots, time and money

An activation to ± 0.005 needs 10 000 shots. The 32-bin bit-count histogram is estimated from the shots in which the output fired ($P(\text{out} = 1) = 0.487$ of them), so the plug-in KL is biased upward by $(K-1)/(2 \cdot 0.487 \cdot N) = 31.8/N$ with $K = 32$; reaching the trained model's 5.5×10^{-4} needs 58 000 shots, rounded to a 64 000-shot job. The ECG PSNR is capped by shot noise at $10 \log_{10}(4N)$, so 2 500 shots per sample cap it at exactly 40 dB — the published Table I value — and a 1 800-sample trace at 25 000 shots is 4.5×10^7 shots: ECG is a simulator experiment. Machine time per job is milliseconds to two seconds on the superconducting machines, where the per-job overhead dominates, but 72 s (Flappy) and 32 minutes (bit-count) on IonQ Forte, where circuit time does.

Prices retrieved 20 August 2026 are in Table IV. Training on hardware is out of the question: 2 160 evaluations at 64 000 shots are 1.4×10^8 shots, 48 days of Forte time or \$11 M. Fine-tuning the six output angles (90 evaluations at 4 000 shots) is 1.8 hours on Forte, about 45 minutes on Fez. Live play is equally out: 1 500 frames $\times$ 4 000 shots $\times$ 17.9 ms is 30 hours per episode on Forte. The falsifiable hardware observable is instead the activation map of Fig. 6 on a grid of observations, 16 jobs.

4. Recommendation

Run the 7-qubit Flappy activation map first: on IBM's free plan to debug the pipeline (readout calibration, then zero-noise extrapolation by gate folding, both validated offline against the noise model), then on IonQ Forte for the confirmation (\$1 300 for a 4×4 grid at 1 000 shots), and on Quantinuum H2-1 for the headline number if an allocation exists. The bit-count histogram is feasible on the H-series and Forte as a *shape* demonstration but not as a KL comparison: device error puts the KL floor at 10^{-2}–10^{-1}, two orders above the simulated 5.5×10^{-4}. Iris cannot be settled yet. What these runs prepare is a uniform superposition over the 12–14 *distinct* training codes, not over the 120 samples an earlier draft assumed; the sparse state-preparation construction of Gleinig and Hoefler [4] has leading term $k \cdot n = 144$–168 two-qubit gates, giving $F \approx 0.83$ on H2-1 and ≈ 0.75 with the network — but its constant is unknown, and the dense alternatives (8 168 gates by Shende–Bullock–

TABLE IV. Cost per job (file `numbers.md`, section "cost"). IBM: \$1.60 per QPU second plus a 30 s per-job overhead, both assumptions (Appendix H); Quantinuum: the vendor's credit formula converted at the Azure plan rate; Braket: published per-shot prices.

Machine	Price model	Flappy, 4 000 shots	bit-count, 64 000 shots
IBM Fez	\$1.60/s; 52 ms resp. 1.8 s of QPU	\$0.08 (\$48 with overhead)	\$2.85 (\$51)
IBM Open plan	10 min per 28 days, free	\$0	\$0
Quantinuum H1/H2	credits $= 5 + (N_1 + 10N_2 + 5N_m)C/5000$	323 credits ≈ \$4 030	11 920 credits ≈ \$149 000
IonQ Forte (Braket)	\$0.30 + \$0.08/shot	\$320	\$5 120
IQM Garnet (Braket)	\$0.30 + \$0.00145/shot	\$6.10	\$93

Markov synthesis, ≈ 3 925 by Plesch–Brukner) are hopeless. An exact answer needs an actual sparse state-preparation circuit, which the repository does not build. For a live submission the code still lacks credentials, the vendors' own routing, asynchronous job handles, and readout-unfolding and gate-folding utilities.

Appendix F: The shot-noise programme

What limits a hardware demonstration is the number of measurements, through two mechanisms: a thresholded controller flips at a rate $\Phi(-2\delta\sqrt{N})$, and a histogram loss estimated from N shots is biased upward by $(K-1)/(2\,P(\text{out}=1)\,N)$ while discarding the half of the shots in which the output did not fire. Six remedies were proposed and three implemented. The others — a joint-distribution loss or amplitude amplification so that all shots are used, bias-corrected KL estimators, and amplitude estimation — are designed but not implemented; the first is the natural next step, since the estimator bias is what now limits sampled training. All runs here are noiseless state-vector simulations plus sampling.

1. Margin-aware training: negative

If every frame had $\delta \geq 0.1$ instead of the median 0.019, the shots per frame would fall by $(0.1/0.019)^2 \approx 25\times$ at no noiseless cost. A penalty $w \times 0.01 \times \max(0, 1-\delta/m)$ per played frame was added — a fraction $w < 1$ of the 0.01 reward for surviving that frame, so surviving always beats dying confidently — and the controller fine-tuned for 25 sweeps with $m = 0.1$, $w \in \{0.25, 0.5, 1\}$ (run `flappy_shots/20260821T084601Z`). An earlier attempt with a *mean* hinge of weight 20, trained from scratch, collapsed to a policy that died in 30 frames with perfectly confident decisions (run `flappy_shots/20260821T075048Z`): any penalty that can outweigh the per-frame reward selects for dying early.

The margin quantiles hardly move (Fig. 15, left; median 0.019 → 0.021–0.024, tenth percentile 0.004 → 0.006). All three fine-tuned controllers still play perfectly on exact probabilities, but under shot noise they play *worse* than the original at $w \geq 0.5$ (50 % and 80 % perfect games at 10 000 shots against 100 %). The near-boundary frames are those in which the bird hovers at the boundary, a property of the game's dynamics and not of the surface's steepness, so penalising them moves the boundary rather than steepening it — and a 17-parameter network has no spare capacity to steepen its surface anyway.

2. Common random numbers

A slope from N independent shots per probe with variance σ^2 has variance $\sigma^2/2c^2$, at $c = 0.2$ some $12.5\sigma^2$, which is why finite differences on a sampling backend stall. Feeding the same uniform random numbers to both probes gives only a 1.3× reduction, because a parameter change moves probability between

distant bit strings and shifts every later bin boundary. What works is the *maximal coupling*: sample the first probe, then for the second keep each outcome i with probability $\min(1, p_{\text{new}}(i)/p_{\text{ref}}(i))$ and redraw otherwise from $(p_{\text{new}} - p_{\text{ref}})_+$. The second sample has exactly the right distribution and only $N \cdot \text{TV}(p_{\text{new}}, p_{\text{ref}})$ shots change; since $\text{TV} \propto c$, the slope's variance is $O(1/(cN))$ rather than $O(1/(c^2N))$, a gain of order $1/c$. Coupling reduces the variance of the slope, not the bias of a non-linear loss. On hardware the analogue is to interleave the $\pm c$ circuits in one job so they share the same drift.

Measured on one slope of the bit-count loss at 400 shots per probe, the reduction is $\times 2$–3 at $c = 0.2$–0.4 and $\times 22$–29 at $c = 0.025$–0.05, both estimates unbiased for the exact slope. Training the 11-qubit network on a sampling backend (run `bitcount_shots/20260821T084558Z`) gives, after 20 sweeps, an exact KL improving from 0.080 to 0.059 at $N = 1\,000$ shots per evaluation, from 0.050 to 0.014 at 4 000, and at 16 000 the two are within the seed-to-seed spread (0.029 against 0.033) — all an order of magnitude above the exact-probability result (0.0009). The likely reason is the loss itself: a plug-in KL of a 32-bin histogram from about $N/2$ usable shots, biased by $31.8/N$, with the reached KL tracking one to three times that bias. Three seeds, one of which stalls at 0.014 even on exact probabilities, cannot separate estimator from optimiser; a bias-corrected or joint loss under paired initialisations is the test, and it has not been run. The schedule used ($c_0 = 0.2$ decaying only to 0.15 over 20 sweeps) also keeps the optimiser where the reduction is smallest.

3. Sequential decisions: positive

A fixed budget wastes most of its shots: a frame with $\delta = 0.3$ is decided correctly after a dozen. Wald's sequential probability ratio test [5] between $P(\text{out} = 1) = \frac{1}{2} \pm \delta_0$ draws shots in blocks, computes after n shots with k ones the log-likelihood ratio $\Lambda = (2k - n) \log \frac{1+2\delta_0}{1-2\delta_0}$, and stops when $\Lambda \geq \log \frac{1-\beta}{\alpha}$ (flap) or $\Lambda \leq \log \frac{\beta}{1-\alpha}$ (do not), up to a cap after which the majority decides. Beyond δ_0 its error probability is at most about α ($= \beta = 5\,\%$ here); nearer the boundary it is slow and unbounded.

On the same 20 held-out games (Fig. 15; Table V) the rule with $\delta_0 = 0.01$ and cap 10 000 wins *every* game at a mean of 2 191 shots per frame, against the 10 000 a fixed budget needs for the same result — $4.6\times$ fewer — and with $\delta_0 = 0.005$, cap 4 000 it matches the fixed 4 000-shot result (85 %) at 2 916. Setting δ_0 at the median margin (0.02) fails (30 % perfect at 822 shots): the frames that decide a game are the near-boundary ones, so the indifference margin must sit below *their* margins, the tenth percentile of 0.004–0.006. The rule needs no change to the network and no retraining.

Appendix G: What the reproduction learned

1. **The central claims reproduce.** Activation surface, bit-count histogram and convergence, Iris overfitting contrast, perfect Flappy play on a clone, ECG denoising within 2 dB on a synthetic signal — though on the authors' own recording only the one-neuron cell of Table I reproduces, the two- and three-neuron prototype cells falling 2.5–3.2 dB short (2.2–3.8 dB on the full recording) (Appendix C 4). What does not: the weight picture of Fig. 2 — scored over 20 seeds against a criterion fixed in advance, trained prototype neurons are far more selective than classic ones but do not cover the data better, and their angles do not decode to data patterns (Appendix C 2). The iterative baseline of Fig. 1 was a second such failure until it turned out to have been graded with the wrong quantity — a scheme that trains one input at a time moves the per-input activations, not the superposition distribution of Eq. (B2), and the prototype block leaves the input register entangled with the hidden qubits — and in its own observable it reaches $\approx 10^{-3}$ on four seeds of six (Appendix C 1). Grade a scheme in the observable it optimises.

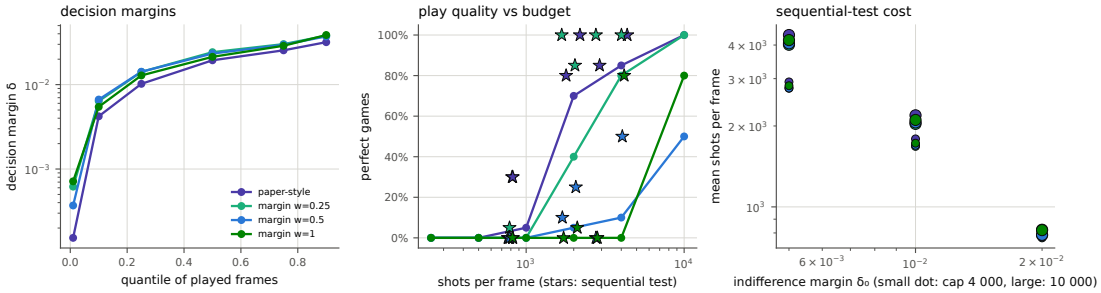


FIG. 15. Flappy under shot noise, remedies 1 and 3. Left: decision-margin quantiles of the frames played by the original and the three margin-aware controllers. Middle: perfect games against shots per frame for a fixed budget (lines) and for the sequential test (stars, plotted at the mean number of shots it actually spent). Right: mean shots per frame of the sequential test against its indifference margin δ_0. Run `flappy_shots/20260821T084601Z`.

TABLE V. Flappy, original controller, 20 held-out games: fixed shot budgets against the sequential test ($\alpha = \beta = 0.05$, blocks of 20 shots). "Flips" are frames decided differently from the exact rule. Run `flappy_shots/20260821T084601Z`.

Decision rule	perfect games (%)	median pipes	flips (%)	mean shots/frame
fixed $N = 1\,000$	5	23	8.36	1000
fixed $N = 2\,000$	70	110	7.40	2000
fixed $N = 4\,000$	85	110	6.09	4000
fixed $N = 10\,000$	100	110	4.47	10 000
sequential, $\delta_0 = 0.02$, cap 10 000	30	88	7.19	822
sequential, $\delta_0 = 0.01$, cap 4 000	80	110	6.12	1790
sequential, $\delta_0 = 0.005$, cap 4 000	85	110	5.98	2916
sequential, $\delta_0 = 0.01$, cap 10 000	100	110	5.06	2191
sequential, $\delta_0 = 0.005$, cap 10 000	100	110	4.50	4358

2. **Reading figures is part of reproduction.** The bit-count target (ones + 1)/112, the Iris label weighting $(\ell + 1)^2$ and the angle encoding $\theta = 2 \arcsin \sqrt{s}$ were all read off figures and then confirmed to the figure's precision. Where a figure was over-read — the Iris split ratio — an independent review caught it and the justification was withdrawn (Appendix C 2).

3. **The prototype network's advantage under noise is inherited, not intrinsic.** Both networks have the same two-qubit gate count and the noise displaces both equally; the prototype wins under noise because it fits better without it.

4. **Gate noise at published levels is not the obstacle; shots are.** Every 7–11-qubit network survives every vendor's calibrated model with a qualitatively intact result, while the Flappy controller needs 10^4 shots per frame at a fixed budget and the bit-count loss is biased by $31.8/N$. The one remedy that pays now is the sequential rule (4.6× fewer shots, no retraining); margin-aware training does not work for a 17-parameter network; common random numbers reduce the optimiser's noise by $1/c$ but leave the estimator's bias, the likely and as yet untested limit of sampled training.

5. **Compiled circuits are 5–15× more noise-sensitive than logical ones.** Two native gates per controlled rotation, resynthesised single-qubit gates and lattice routing all cost; a complete-bipartite

network is the worst case for a lattice, so the vendor's own compiler must be used for any superconducting submission. Trapped ions are the natural home of this architecture, at the price of circuit time.

6. **An optimiser that keeps a stale incumbent turns a harmless change into a negative result.** The "rotating episodes break training" finding was an artefact of comparing losses measured on different episode batches, because the optimiser kept its incumbent's loss from the previous batch; re-scoring the incumbent on each batch restores 25/25 perfect games (runs `flappy/batch0`, `flappy/batch5`). Any non-stationary objective needs this.

7. **Penalties must be priced in the currency of the reward.** A margin hinge able to outweigh the per-frame survival reward produced a confidently dead controller; the per-frame form, kept below that reward, does not.

8. **Provenance has to be mechanical.** The first hardware plan contained a dozen wrong numbers: a state-preparation gate count off by a factor of two, a reversed inequality, costs computed by hand. The fix was a script that emits every number and refuses to run if any calibration input lacks a source; this appendix quotes only that script.

9. **A read-out calibrated on the evaluated set is not a held-out accuracy.** The Iris labels are rounded after one overall scale is fitted; fitted on the test set itself, it let three removed test flowers change the predicted labels of most of the rest (94 % → 39 % on one seed). Only the 20-seed sweep exposed it; one seed would not have. Calibrate on the training data, and state the pass criterion before running the seeds.

Appendix H: Choices the submission leaves open

Each entry is marked `[ASSUMPTION]` where it enters the code.

Optimiser schedule.: $a_n = a_0/(n+1)^{0.602}$, $c_n = c_0/(n+1)^{0.101}$, geometric cooling; one iteration is a coordinate sweep. Bit-count: $a_0 = 20$, $c_0 = 0.2$, $T_0 = 0.02$, cooling 0.7, 20 sweeps. Flappy: $a_0 = 0.4$, $c_0 = 0.25$, $T_0 = 0.05$, cooling 0.9, 25 sweeps after 300 random restarts. Iris: $a_0 = 2$, decay exponent 0.3, $c_0 = 0.1$, 40 sweeps. ECG: 40 sweeps per restart, 8 restarts, best by training MSE.

Iris ambiguous codes.: By default, samples sharing a 12-bit code are merged and their labels averaged. The original preprocessing removed "binary patterns which were assigned to two classes" (e-mail, 2026-08-21); the reimplementation's drop policy does that on both splits after encoding, so the test accuracy under it is conditional on the pattern being unambiguous (seed 0: 97.6 % on the 82 surviving test samples against 95.0 % on all 100; over 20 seeds 96.4 % against 94.1 % with the training-set read-out scale); both numbers are reported. Which split was filtered, and whether before or after the k-means encoding, is not stated.

Iterative baseline.: One *elementary* Kiefer–Wolfowitz step per iteration — one coordinate, one randomly chosen input, four objective evaluations — on the squared error of that input's activation; only the KW step is offered to the Metropolis rule; fixed schedule $a = 2$, $c = 0.05$, $T = 0$; 10^5 iterations with the per-input D_{KL} monitored every 1 000. Identifying the published plotted iteration with this elementary step is itself the assumption that makes its ≈ 120 iterations and these 10^5 steps comparable.

Angle encoding.: $\theta = 2 \arcsin \sqrt{s}$, pinned by a printed angle in the companion paper; $\theta = \pi s$ tested as an ablation.

KL regularisation.: Model probability floored at 10^{-12} inside the logarithm.

Iris.: Stratified one-third training split; flowers sharing a code are merged and their labels averaged; the KL is normalised over all 4 096 input strings, since restricting it to the dataset's codes lets the network exploit interference between examples and reach a training loss of 10^{-5} while reading out at chance; label weight $\propto (\ell + 1)^2$; labels predicted by nearest rounding of the read-out weight, after one overall scale (the class weights $1 : 4 : 9$ are known only up to a constant) is fitted by least squares in log space. Fitted on the evaluated set, as in the submission's wording, the scale makes a flower's label depend on the other flowers in the set; fitted once on the training set it does not. Both are reported; the second is the one quoted as a held-out accuracy.

Weight-picture criterion.: The t-SNE picture of Fig. 2 is replaced by a firing-based coverage and selectivity score (z-scores against 1 000 untrained layers from the initialisation) and an angle-based covering score ρ; the pass criterion (20 seeds, both firing scores positive and above the classic layer's at $p < 0.01$, median $\rho \geq 0.5$) was fixed before the seeds were run.

Flappy.: Clean-room clone: 288×512 screen, flap impulse -7, gap 120, pipe spacing 180, gravity 1; reward 0.01 per frame plus 1 per pipe; tracking term $|\text{height} - \text{gap}|$ with weight 1; 5 training episodes of 1 500 frames, 20 held-out seeds 5–24, 5 000-frame cap (110 pipes); decisions cached on a 65×65 grid.

ECG.: Three signals: a synthetic McSharry-form signal tuned to the published trace with Gaussian noise $\sigma = 0.02$; MIT-BIH record 100; and the authors' own recording, of which the first 1 800 samples — the published window — are used, with the full 7 700 as a check. On that recording the noise is the recording's own ($\sigma = 0.035$, input 27.0 dB) and not a choice. Assumed for all three: mirror-padded edges; first half train, second half test; 8 optimiser restarts per cell with the best taken by training MSE; PSNR with R the range of the clean signal; SSIM both whole-signal and in a 7-sample sliding window.

Noise model.: Depolarising at the vendor's average gate error, relaxation during gates and as-soon-as-possible idling, readout confusion; medians where vendors publish per-qubit tables; no crosstalk, since no vendor publishes a usable number. Quantinuum's idle error is charged as $(1 - m)^{n_{\text{qubits}} \cdot \text{depth}}$ from its memory error per depth-1 time; IQM's and Rigetti's unpublished one-qubit gate durations are set to 40 ns.

Fidelity estimate.: First-order product of per-gate survival probabilities; readout factor $(1 - (p_{01} + p_{10})/2)^n$.

Costs.: IBM \$1.60 per QPU second (2022 launch figure, corroborated by a secondary \$96/min report) plus 30 s per-job overhead; Quantinuum \$12.5 per credit (Azure Standard plan divided by its allotment); Braket and Azure per-shot prices as retrieved on 20 August 2026; Quantinuum's credit formula with $N_m = 2\times$ qubits.

State preparation.: Dense counts $2^{n+1} - 2n$ (Shende–Bullock–Markov) and $\approx \frac{23}{24} 2^n$ (Plesch–Brukner); sparse *leading term* $k \cdot n$ of an $O(kn)$ construction [4] with k the 12–14 distinct training codes actually prepared, its constant unknown.

Compilation.: Lattice machines are compiled onto a synthetic heavy-hex or square patch of the circuit's size with a greedy router; Google's and Rigetti's gate sets are rebased through a generic fallback. Idle relaxation is charged over each qubit's schedule gaps only.

Shot-noise programme.: Margin penalty $w \cdot 0.01 \cdot \max(0, 1 - \delta/m)$ per frame with $m = 0.1$; sequential test with $\alpha = \beta = 0.05$ and blocks of 20 shots; common random numbers by maximal

coupling to the first probe of each coordinate.

Appendix I: Provenance of the calibration data

Every calibration number used in Appendices D and E is stored in a device file with a provenance block per field: source URL, kind of source (vendor calibration page, vendor paper, third-party snapshot), retrieval date and a verbatim quotation of the sentence or table row the number was read from. The hardware-plan script joins these blocks to every field it reads and fails if any is missing; the full join, one row per device and field with quotations, is the file `numbers.md` of run `hardware-plan/20260821T084554Z`. In brief:

IBM Fez (Heron r2).: One coherent calibration snapshot, backend version 1.2.8 of 2025-02-26, shipped as `props_fez.json` in `qiskit-ibm-runtime`'s fake provider; medians over 156 qubits and 352 directed CZ edges computed from that table. The file records the same chip drifting between CZ error 2.85×10^{-3} (2024-07) and 3.90×10^{-3} (2025-02).

Quantinuum H1-1, H2-1.: The vendor's performance-validation page: mean two-qubit and one-qubit infidelities, SPAM error, "Memory Error per Depth-1 Circuit Time". No gate durations, no T_1/T_2.

IonQ Forte.: The vendor's published specification: average gate fidelities, gate durations, $T_1 = 55\,\mathrm{s}$, $T_2 = 1\,\mathrm{s}$.

Google Willow.: Supplementary information of the Willow paper (*Nature*, 2024): gate errors, coherence times, cycle time. Not publicly accessible; the preset exists for simulation only.

IQM Garnet.: Vendor data for gate errors and readout; T_1 and T_2 (echo) from a third-party calibration snapshot, two epochs recorded (December 2025, March 2026), with the caveat that T_2 without echo is $\approx 8.5\,\mu\mathrm{s}$.

Rigetti Ankaa-3.: Only the two-qubit (iSWAP) error 10^{-2} is published for the universal gate; the 99.5 % headline refers to the fSim gate. No one-qubit error, no readout error, no T_1 or T_2.

The per-field table is not reproduced here — some sixty rows with quotations — but it is the ground truth every number in this appendix rests on, and it is one command away in the repository at `https://github.com/tobiasosborne/prototype-learning`.

One experimental input is not calibration data: the ECG recording of Appendix C 4, supplied by the authors by e-mail on 21 August 2026 as two MATLAB files. It is the authors' data and is not redistributed; instead the loader writes the SHA256 digest of each file into the `meta.json` of every run that uses it, so runs `ecg/20260918T132924`, `ecg/20260925T153739Z-mailed1800` and `ecg/20260925T165741Z-mailedfull` identify the input exactly without carrying it.